

INF1103: Programming Fundamentals with DevOps

Week 3: Procedural Design and Functions



Learning Outcomes

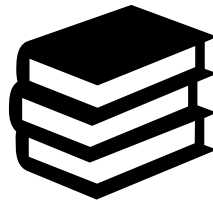
- Understand why functions are used in software development
- Write and use your own Python functions
- Pass information into and out of functions
- Break a complex problem into smaller manageable pieces
- Build simple real-world solutions using



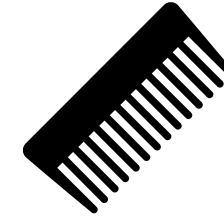
Calculate Final Order Price



Item 1
S\$ 20



Item 2
S\$ 50



Item 3
S\$ 10

Calculate Final Order Price



Order 1 → Copy Logic

Order 2 → Copy Logic

Order 3 → Copy Logic

Order 4 → Copy Logic

Order 5 → Copy Logic

...

Order 1,000,000 → Copy Logic

```
#Item 1
price = 20
discount = price * 0.10
subtotal = price - discount
gst = subtotal * 0.09
final_price = subtotal + gst
print(final_price)
```

```
#Item 2
price = 50
discount = price * 0.10
subtotal = price - discount
gst = subtotal * 0.09
final_price = subtotal + gst
print(final_price)
```

```
#Item 3
price = 10
discount = price * 0.10
subtotal = price - discount
gst = subtotal * 0.09
final_price = subtotal + gst
print(final_price)
```

Calculate Final Order Price



- ✗ Same logic repeated many times
- ✗ Difficult to maintain
- ✗ Easy to make mistakes
- ✗ Imagine doing this for millions of Shopee orders every day

```
#Item 1
price = 20
discount = price * 0.10
subtotal = price - discount
gst = subtotal * 0.09
final_price = subtotal + gst
print(final_price)

#Item 2
price = 50
discount = price * 0.10
subtotal = price - discount
gst = subtotal * 0.09
final_price = subtotal + gst
print(final_price)

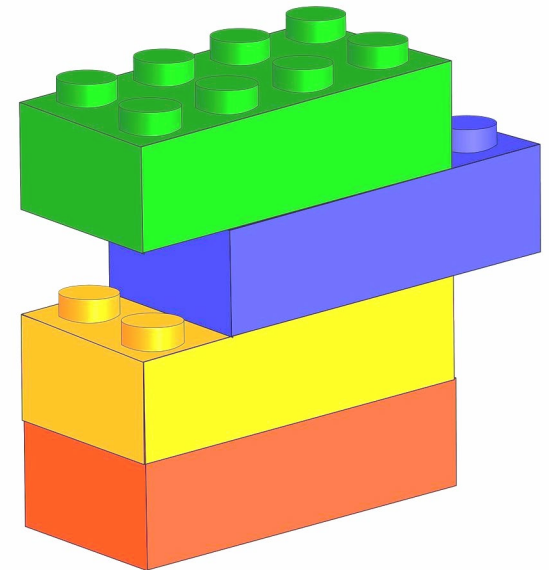
#Item 3
price = 10
discount = price * 0.10
subtotal = price - discount
gst = subtotal * 0.09
final_price = subtotal + gst
print(final_price)
```

Why do we need functions?

- Avoid repeating code
- Improve readability
- Break problems into smaller parts
- Easier testing and maintenance
- Encourages logical thinking

What is a function?

Function is a block of organized, reusable code that is used to perform a single, related action.



Function Structure

```
def function_name( arguments ):
    '''function_docstring'''
    function_suite
    return [expression]
```

- **Function_name**
 - The identifier of the function
- **Arguents**
 - Input
 - Formal Arguments
 - Actual Arguments
- **Function_docstring**
 - Comment your codes
 - Optional
- **Function_suite**
 - Block of code
 - **indented**
- **Return value**
 - Optional
 - Exits a function
 - Passing back an expression to the caller
 - Output



Function Arguments

- A function can have 0 or multiple arguments
- When you call one function, it must have the same number of arguments as defined

Function Arguments

- Formal Arguments
 - A place to get input value
 - A name used inside a function
- Actual Arguments
 - True value

```
def greet(name): # formal argument  
    print("Hello", name)
```

```
greet("Nisha") # actual argument
```

Positional Arguments

- Positional Arguments
 - Passed to the function in the order in which they are defined.
 - Order matters, arguments are matched by position.

```
def create_order(customer, product, quantity):  
    print(customer, "ordered", quantity, product)  
  
create_order("John", "Laptop", 2)  
create_order("Laptop", "John", 2)
```

John ordered 2 Laptop
customer → "John"
product → "Laptop"
quantity → 2

Laptop ordered 2 John
customer → "Laptop"
product → "John"
quantity → 2

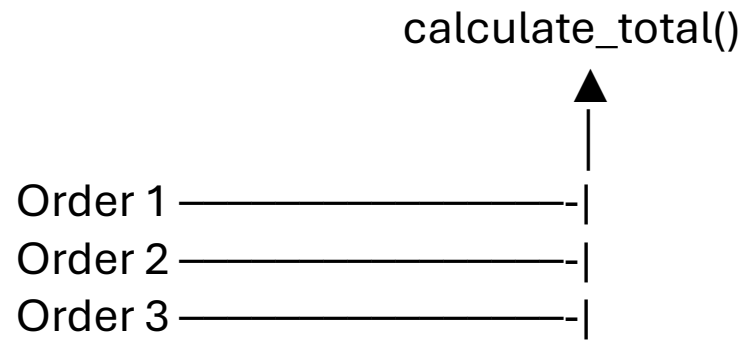
Keyword Arguments

- Keyword Arguments
 - Passed to the function by explicitly naming each parameter.
 - Order doesn't matter, parameters are matched by name.
 - To enhance the **robustness** and **readability** of the program
 - Especially when there are many parameters

```
def create_order(customer, product, quantity):  
    print(customer, "ordered", quantity, product)  
  
create_order(customer="John", product="Laptop", quantity=2)  
create_order(quantity=2, product="Laptop", customer="John")
```

John ordered 2 Laptop
customer → "John"
product → "Laptop"
quantity → 2

Calculate Final Order Price



```
def calculate_total(price):  
    discount = price * 0.10  
    subtotal = price - discount  
    gst = subtotal * 0.09  
    return subtotal + gst  
  
print(calculate_total(20))  
print(calculate_total(50))  
print(calculate_total(10))
```



Tuple

- Python data structure used to store multiple values in a single variable.
- Using circular brackets
- Immutable
- Unpacking

```
numbers = (10, 20, 30)  
print(numbers)
```

```
firstRank,secondRank,thirdRank = numbers
```

Function return values

- When Python reaches a return statement, the function immediately stops.
- Return Different data types
- If no return statement, automatically returns None.
- Return multiple values as a Tuple

```
def calculate_discount(price):  
    discount = price * 0.10  
    return discount
```

```
def calculate(a, b):  
    return a + b, a * b  
sum_value, product_value = calculate(4, 5)  
print(sum_value)  
print(product_value)
```

Function Arguments

- Suppose we want to calculate the total price of items in a Shopee cart.

```
def calculate_total(item1, item2, item3):  
    return item1 + item2 + item3  
  
print(calculate_total(20, 15, 30))
```

- What if we don't know how many values will be passed to a function? What if the customer buys:
 - 2 items?
 - 5 items?
 - 10 items?
 - Would we need different functions?

Function Arguments

```
def function_name(*args):  
    pass
```

- Accepts any number of positional arguments
- Stores them in a Tuple
- Useful when the number of inputs is unknown
- Common in real-world applications such as shopping carts, data processing, and reports

Function Arguments

- `*args` collects multiple arguments into a tuple

```
def calculate_total(*prices):  
    print(prices)  
  
print(calculate_total(20, 15, 30))  
Print(calculate_total(10,15,30,40,50))
```

- The number of items varies for every customer.
- Instead of creating different functions, `*prices` handles all cases.



Function Arguments

****kwargs**

- Allows a function to accept any number of **keyword arguments**.
- Pack keyword arguments into a dictionary.

Scope of a variable

- Local Variable

- Variables defined within a function can only be accessed and used inside that function.
- They "come to life" when the function is called and "die" (or go out of scope) when the function execution is completed.
- `calculate_discount(100)`
 - `price = 100`
 - `discount = 10`
 - Return 10
 - Function ends -> discount disappears

```
def calculate_discount(price):  
    discount = price * 0.10  
    return discount  
print(calculate_discount(100))  
print(discount)
```

TypeError: name 'discount' is not defined

Function as machines

- Input $\rightarrow$ Function $\rightarrow$ Output
- Example: Vending Machine:
 - Money + Selection $\rightarrow$ Machine $\rightarrow$ Drink
- Python Function:
 - Data $\rightarrow$ Function $\rightarrow$ Result



Functions in Industry

- Modern systems are built from thousands of functions:
 - `check_stock()`
 - `calculate_total()`
 - `process_payment()`
 - `send_confirmation()`

Procedural Design

Customer Login



Verify User



Check Inventory



Process Payment



Create Order

All system start with
processes and workflows.

Why Do We Need Procedural Design?

- Imagine I ask you to build a Shopee checkout system.
- How do you know what code to write?
 - Before coding, we need a plan.

Why do we need Procedural Design

Customer Login



Verify User



Check Inventory



Process Payment



Create Order

- All system start with processes and workflows.
- Only after the workflow is understood do developers implement the code.

Why do we need Procedural Design

- Understand the problem clearly
- Break large problems into smaller tasks
- Reduce logical errors
- Make coding easier
- Support the creation of reusable functions
- Reflect how real software is developed in industry

Procedural Design

- Problem → Procedure → Code
- Think about inputs, outputs and steps before coding.
- Process of planning the steps needed to solve a problem before writing the code.



Poor vs Good Design

- Poor: One giant block of code
- Good: `verify_user()`, `check_stock()`, `process_payment()`

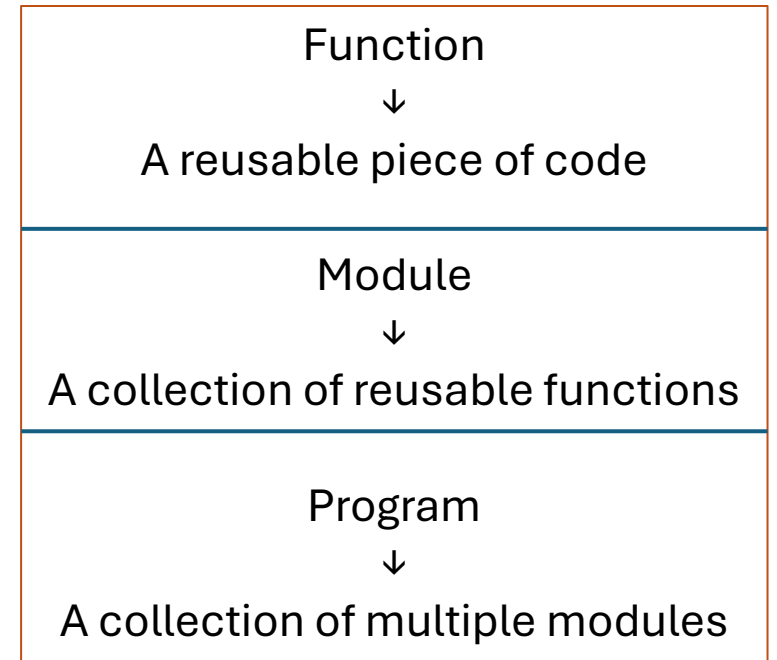
Task for the day!!

- Design a parking-charge system:
 - First 2 hours free
 - Then \$2 per hour
- Write the procedure first, then code it.



Modules

- Put related functions together.
- A module is a file containing definition of functions, classes, variables, constants or any other Python object that can be **reused**
- Python has the **import** keyword to load a module.



Modules: Create your own

- **Creating module:**
 - Write one or more functions in a text file
 - Save it with a .py extension
 - Save in the same directory as other scripts
 - Give one descriptive name
- **Using a module:**
 - `import module_name [as simple_name]`
 - Without .py extension
 - Use a function:
 - `module_name.fun_name()`

Modules

```
def calculate_total():  
    pass  
  
def apply_discount():  
    pass  
  
def calculate_gst():  
    pass
```

Without Modules:

- Everything is in one file
- As programs grow larger, this becomes difficult to manage

billing.py

```
def apply_discount(price):  
    return price * 0.9  
def calculate_gst(price):  
    return price * 1.09
```

```
import billing  
  
price = 100  
price = billing.apply_discount(price)  
price = billing.calculate_gst(price)  
  
print(price)
```

With Modules:

- Organizes code
- Promotes reuse
- Makes maintenance easier



Python built-in Modules

- math
- random
- datetime
- statistics
- os
- json
- urllib2
- csv



Functions as Objects

- A function can be:
 - Assigned to a variable
 - Passed as an argument
 - Returned from another function
 - Stored in data structures

Functions as Objects

- A function can be:
 - Assigned to a variable
 - Passed as an argument
 - Returned from another function
 - Stored in data structures

```
def greet():  
    print("Hello")  
  
message = greet  
  
message()
```

Functions as Objects

- A function can be:
 - Assigned to a variable
 - Passed as an argument
 - Returned from another function
 - Stored in data structures

```
def square(x):  
    return x * x  
  
def apply_function(func, value):  
    return func(value)  
  
print(apply_function(square, 5))
```

Task for the day!!

- Smart Vending Machine

- Rules:

- Create a function called `dispense_drink()` that accepts a drink name and displays the correct message.
 - If the drink is "Coke" → Print "Dispensing Coke"
 - If the drink is "Water" → Print "Dispensing Water"
 - If the drink is "Juice" → Print "Dispensing Juice"
 - Otherwise → Print "Drink Not Available"

- Challenge: Modify the function so that it also keeps track of the number of drinks dispensed





Takeaway!!

- Think before you code
- Design before you implement
- Decompose before you solve
- Reuse instead of repeat
- Focus on logic, not just syntax

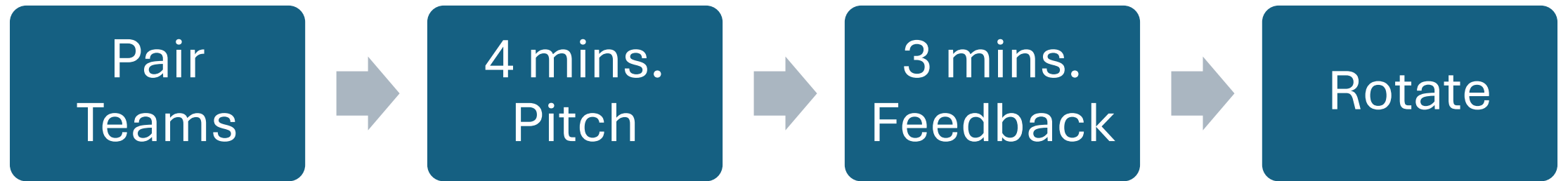
Good programmers don't write code first. They design solutions first, then use functions to implement them.



Lab this week!!

- 1st hour: Speed Projecting
- 2nd Hour: Deliberating and Work on Labsheet

Speed Projecting: How it works



Your 4-Minute Pitch

- The problem & why it is real
- Data flow: Input -> AI -> Output
- One rule that logic_manager applies
- What will be the expected output of Part 1

A large, dark blue, cloud-like shape with a black outline is positioned on the right side of the slide. It contains white text.

All at design
phase no coding
done yet

Give Useful Feedback

? Is the AI doing real work?

? Is it too big or small for 6 weeks?

? Do the proposed application's inputs and outputs align logically with its intended functionality?

Why Speed Projecting?

- Catch weak ideas early
- Check if AI adds real value
- Get feedback before starting implementation



Easier to pivot in Week 3 than panic in Week 5

Thanks!!!

